

Guía SQL de Básico a Experto

Usando la base de datos Sakila - teoría, ejemplos, ejercicios y solucionario

Qué incluye	Ruta completa de estudio, consultas reales, agregaciones, castings, fechas, joins, CTEs, ventanas y escenarios de negocio.
Enfoque	Cada tema explica el concepto, muestra ejemplos y cierra con ejercicios prácticos y respuestas orientadas a entrevistas y trabajo real.
Base	Sakila: películas, clientes, rentas, pagos, inventario, categorías, tiendas, ciudades y países.

Sugerencia de uso: intenta resolver primero sin ver el solucionario. Después compara tu respuesta, revisa si tu SQL es claro, correcto y eficiente, y vuelve a escribirlo con mejores alias y formato.

Referencia rápida de tablas comunes: **actor**, **film**, **category**, **film_category**, **customer**, **payment**, **rental**, **inventory**, **staff**, **store**, **address**, **city**, **country**.

1. SELECT, WHERE, ORDER BY y LIMIT

Este bloque construye la base. Aprendes a elegir columnas, filtrar filas, ordenar resultados y limitar la salida. También es donde se empieza a escribir SQL legible: sangría, alias y condiciones claras.

```
SELECT title, release_year, rating
FROM film
WHERE rating = 'PG-13'
ORDER BY title
LIMIT 10;
```

- Selecciona solo lo necesario; evita SELECT * en análisis reales si no necesitas todas las columnas.
- Combina filtros con AND y OR, pero usa paréntesis cuando la lógica pueda ser ambigua.
- ORDER BY puede ir por varias columnas.

Ejercicios

1. Muestra actor_id, first_name y last_name de todos los actores.
2. Lista las películas PG ordenadas por duración de mayor a menor.
3. Muestra las primeras 15 películas con rental_rate = 0.99.
4. Obtén los clientes activos con nombre y apellido.
5. Lista las películas con duración entre 120 y 150 minutos.
6. Muestra los actores cuyo apellido sea DAVIS.
7. Lista las películas cuyo rating sea G o PG.
8. Muestra las películas que no sean NC-17.

Solucionario

1.

```
SELECT actor_id, first_name, last_name FROM actor;
```

2.

```
SELECT title, length FROM film WHERE rating = 'PG' ORDER BY length DESC;
```

3.

```
SELECT title, rental_rate FROM film WHERE rental_rate = 0.99 LIMIT 15;
```

4.

```
SELECT customer_id, first_name, last_name FROM customer WHERE active = 1;
```

5.

```
SELECT title, length FROM film WHERE length BETWEEN 120 AND 150 ORDER BY length;
```

6.

```
SELECT actor_id, first_name, last_name FROM actor WHERE last_name = 'DAVIS';
```

7.

```
SELECT title, rating FROM film WHERE rating IN ('G','PG') ORDER BY rating, title;
```

8.

```
SELECT title, rating FROM film WHERE rating <> 'NC-17';
```

2. Alias, expresiones, CASE y limpieza de salida

Un analista no solo consulta: también prepara salidas comprensibles. Los alias renombran columnas. Las expresiones crean métricas nuevas. CASE clasifica datos y ayuda a traducir códigos en categorías de negocio.

```
SELECT title AS pelicula,  
length AS minutos,  
CASE  
WHEN length < 90 THEN 'Corta'  
WHEN length < 120 THEN 'Media'  
ELSE 'Larga'  
END AS tipo_duracion  
FROM film;
```

- Usa alias consistentes y sin ambigüedades.
- CASE sirve para segmentar y también para derivar banderas 0/1.
- Puedes combinar columnas con CONCAT para etiquetas legibles.

Ejercicios

1. Muestra title como pelicula y rental_rate como tarifa.
2. Crea una columna nivel_precio con 'Bajo' si rental_rate <= 2.99 y 'Alto' en otro caso.
3. Clasifica las películas en Corta (<90), Media (90-119) y Larga (120+).
4. Concatena first_name y last_name en una sola columna nombre_completo para customer.
5. Crea una bandera es_activo de 1 si active = 1 y 0 si no.
6. Muestra title y una nueva columna costo_total = replacement_cost + rental_rate.
7. Crea una columna rango_rating que diga Familiar para G y PG; Adulto para R y NC-17; Otro para lo demás.
8. Muestra city y country con alias legibles usando join.

Solucionario

1.

```
SELECT title AS pelicula, rental_rate AS tarifa FROM film;
```

2.

```
SELECT title, rental_rate, CASE WHEN rental_rate <= 2.99 THEN 'Bajo' ELSE 'Alto' END AS nivel_precio  
FROM film;
```

3.

```
SELECT title, length, CASE WHEN length < 90 THEN 'Corta' WHEN length < 120 THEN 'Media' ELSE 'Larga'  
END AS categoria_duracion FROM film;
```

4.

```
SELECT customer_id, CONCAT(first_name, ' ', last_name) AS nombre_completo FROM customer;
```

5.

```
SELECT customer_id, first_name, active, CASE WHEN active = 1 THEN 1 ELSE 0 END AS es_activo FROM  
customer;
```

6.

```
SELECT title, replacement_cost, rental_rate, replacement_cost + rental_rate AS costo_total FROM  
film;
```

7.

```
SELECT title, rating, CASE WHEN rating IN ('G','PG') THEN 'Familiar' WHEN rating IN ('R','NC-17')  
THEN 'Adulto' ELSE 'Otro' END AS rango_rating FROM film;
```

8.

```
SELECT ci.city AS ciudad, co.country AS pais FROM city ci JOIN country co ON ci.country_id =  
co.country_id;
```

3. Funciones de texto y patrones

En datos reales es común normalizar nombres, buscar patrones y construir etiquetas. Aquí entran UPPER, LOWER, LENGTH, SUBSTRING, REPLACE, TRIM y LIKE.

```
SELECT first_name,  
UPPER(first_name) AS nombre_mayuscula,  
LENGTH(first_name) AS longitud  
FROM customer  
WHERE first_name LIKE 'A%';
```

- LIKE usa % para múltiples caracteres y _ para un solo carácter.
- TRIM y REPLACE ayudan a limpiar datos.
- Las funciones de texto también sirven en joins de estandarización, aunque conviene usarlas con cuidado por desempeño.

Ejercicios

1. Muestra first_name en mayúsculas para customer.
2. Lista los actores cuyo nombre empiece con A.
3. Lista las ciudades que contengan la letra a.
4. Muestra title y la longitud del título.
5. Extrae los primeros 5 caracteres del título de cada película.
6. Reemplaza la letra A por @ en los nombres de actor.
7. Muestra los apellidos de actor en minúsculas.
8. Encuentra películas cuyo título termine en Y.

Solucionario

1.

```
SELECT first_name, UPPER(first_name) AS first_name_upper FROM customer;
```

2.

```
SELECT actor_id, first_name, last_name FROM actor WHERE first_name LIKE 'A%';
```

3.

```
SELECT city FROM city WHERE city LIKE '%a%';
```

4.

```
SELECT title, LENGTH(title) AS largo_titulo FROM film;
```

5.

```
SELECT title, SUBSTRING(title, 1, 5) AS inicio_titulo FROM film;
```

6.

```
SELECT first_name, REPLACE(first_name, 'A', '@') AS nombre_modificado FROM actor;
```

7.

```
SELECT last_name, LOWER(last_name) AS last_name_lower FROM actor;
```

8.

```
SELECT title FROM film WHERE title LIKE '%Y';
```

4. Agregaciones y GROUP BY

Aquí empieza el análisis real. COUNT, SUM, AVG, MIN y MAX convierten filas en métricas. GROUP BY agrega por categoría, cliente, tienda o cualquier dimensión analítica.

```
SELECT rating,  
COUNT(*) AS total_peliculas,  
AVG(length) AS duracion_promedio  
FROM film  
GROUP BY rating  
ORDER BY total_peliculas DESC;
```

- Toda columna no agregada debe estar en GROUP BY.
- Diferencia entre COUNT(*) y COUNT(columna): la segunda ignora NULL.
- Usa ROUND para resultados más legibles.

Ejercicios

1. Cuenta cuántas películas hay por rating.
2. Obtén el promedio de duración por rating.
3. Calcula el mínimo y máximo replacement_cost por rating.
4. Cuenta cuántos clientes hay por store_id.
5. Suma el total amount pagado por customer_id.
6. Cuenta cuántos pagos se hicieron por staff_id.
7. Obtén el promedio de rental_duration por rating.
8. Cuenta películas por release_year.

Solucionario

1.

```
SELECT rating, COUNT(*) AS total_peliculas FROM film GROUP BY rating ORDER BY total_peliculas DESC;
```

2.

```
SELECT rating, AVG(length) AS promedio_duracion FROM film GROUP BY rating;
```

3.

```
SELECT rating, MIN(replacement_cost) AS min_cost, MAX(replacement_cost) AS max_cost FROM film GROUP BY rating;
```

4.

```
SELECT store_id, COUNT(*) AS total_clientes FROM customer GROUP BY store_id;
```

5.

```
SELECT customer_id, SUM(amount) AS total_pagado FROM payment GROUP BY customer_id ORDER BY total_pagado DESC;
```

6.

```
SELECT staff_id, COUNT(*) AS total_pagos FROM payment GROUP BY staff_id;
```

7.

```
SELECT rating, AVG(rental_duration) AS promedio_rental_duration FROM film GROUP BY rating;
```

8.

```
SELECT release_year, COUNT(*) AS total_peliculas FROM film GROUP BY release_year;
```

5. HAVING y análisis agregado con filtros

WHERE filtra filas antes de agregar. HAVING filtra grupos después de calcular las métricas. Es clave para preguntas de negocio como 'categorías con más de X películas' o 'clientes con gasto superior al promedio'.

```
SELECT customer_id,  
SUM(amount) AS total_pagado  
FROM payment  
GROUP BY customer_id  
HAVING SUM(amount) > 150  
ORDER BY total_pagado DESC;
```

- HAVING puede usar funciones agregadas.
- Una práctica útil: usa WHERE para recortar el universo y HAVING para filtrar el resultado agregado.
- Muchos reportes reales combinan ambos.

Ejercicios

1. Muestra ratings con más de 200 películas.
2. Clientes cuyo total pagado sea mayor a 150.
3. Categorías con más de 60 películas.
4. Tiendas con más de 300 clientes.
5. Ratings cuyo promedio de duración sea mayor a 115 minutos.
6. Clientes con al menos 30 pagos.
7. Actores con más de 25 películas.
8. Películas con más de 30 rentas.

Solucionario

1.

```
SELECT rating, COUNT(*) AS total_peliculas FROM film GROUP BY rating HAVING COUNT(*) > 200;
```

2.

```
SELECT customer_id, SUM(amount) AS total_pagado FROM payment GROUP BY customer_id HAVING SUM(amount) > 150;
```

3.

```
SELECT c.name AS categoria, COUNT(*) AS total_peliculas FROM film_category fc JOIN category c ON  
fc.category_id = c.category_id GROUP BY c.name HAVING COUNT(*) > 60;
```

4.

```
SELECT store_id, COUNT(*) AS total_clientes FROM customer GROUP BY store_id HAVING COUNT(*) > 300;
```

5.

```
SELECT rating, AVG(length) AS promedio_duracion FROM film GROUP BY rating HAVING AVG(length) > 115;
```

6.

```
SELECT customer_id, COUNT(*) AS total_pagos FROM payment GROUP BY customer_id HAVING COUNT(*) >= 30;
```

7.

```
SELECT a.actor_id, a.first_name, a.last_name, COUNT(*) AS total_peliculas FROM actor a JOIN  
film_actor fa ON a.actor_id = fa.actor_id GROUP BY a.actor_id, a.first_name, a.last_name HAVING  
COUNT(*) > 25 ORDER BY total_peliculas DESC;
```

8.

```
SELECT f.title, COUNT(*) AS total_rentas FROM rental r JOIN inventory i ON r.inventory_id =  
i.inventory_id JOIN film f ON i.film_id = f.film_id GROUP BY f.title HAVING COUNT(*) > 30 ORDER BY  
total_rentas DESC;
```

6. JOINS: el corazón del análisis relacional

Los joins conectan el modelo. Con Sakila puedes ir de pago a renta, de renta a inventario, de inventario a película y de película a categoría. Dominar esto te permite responder preguntas reales de ventas, catálogo y operación.

```
SELECT c.first_name, c.last_name, p.amount, p.payment_date
FROM customer c
JOIN payment p ON c.customer_id = p.customer_id
ORDER BY p.payment_date DESC;
```

- Empieza por entender la llave de cada tabla.
- Usa alias cortos y consistentes.
- LEFT JOIN sirve cuando quieres conservar filas aunque no exista coincidencia en la tabla derecha.

Ejercicios

1. Une customer con address para mostrar nombre del cliente y dirección.
2. Muestra película y categoría usando film, film_category y category.
3. Muestra cliente y país de residencia.
4. Lista pagos con nombre del staff que cobró.
5. Obtén actor y película usando actor y film_actor.
6. Cuenta cuántas películas tiene cada categoría.
7. Muestra cada renta con título de película y cliente.
8. Usa LEFT JOIN para mostrar todas las categorías aunque quisieras comprobar si tienen películas asociadas.

Solucionario

1.

```
SELECT c.customer_id, c.first_name, c.last_name, a.address FROM customer c JOIN address a ON
c.address_id = a.address_id;
```

2.

```
SELECT f.title, c.name AS categoria FROM film f JOIN film_category fc ON f.film_id = fc.film_id JOIN
category c ON fc.category_id = c.category_id;
```

3.

```
SELECT cu.first_name, cu.last_name, co.country FROM customer cu JOIN address a ON cu.address_id =
a.address_id JOIN city ci ON a.city_id = ci.city_id JOIN country co ON ci.country_id =
co.country_id;
```

4.

```
SELECT p.payment_id, p.amount, s.first_name, s.last_name FROM payment p JOIN staff s ON p.staff_id =
s.staff_id;
```

5.

```
SELECT a.first_name, a.last_name, f.title FROM actor a JOIN film_actor fa ON a.actor_id =
fa.actor_id JOIN film f ON fa.film_id = f.film_id;
```

6.

```
SELECT c.name AS categoria, COUNT(*) AS total_peliculas FROM category c JOIN film_category fc ON
c.category_id = fc.category_id GROUP BY c.name ORDER BY total_peliculas DESC;
```

7.

```
SELECT r.rental_id, cu.first_name, cu.last_name, f.title FROM rental r JOIN customer cu ON
r.customer_id = cu.customer_id JOIN inventory i ON r.inventory_id = i.inventory_id JOIN film f ON
i.film_id = f.film_id;
```

8.

```
SELECT c.name AS categoria, COUNT(fc.film_id) AS total_peliculas FROM category c LEFT JOIN
film_category fc ON c.category_id = fc.category_id GROUP BY c.name;
```

7. CAST, CONVERT, COALESCE y manejo de tipos

En proyectos reales muchas columnas llegan como texto o con formatos mixtos. Saber castear ayuda a comparar, redondear y construir métricas. COALESCE es clave para tratar nulos sin romper cálculos.

```
SELECT amount,  
CAST(amount AS DECIMAL(10,2)) AS amount_decimal,  
COALESCE(return_date, rental_date) AS fecha_referencia  
FROM rental r  
LEFT JOIN payment p ON r.rental_id = p.rental_id;
```

- CAST transforma el tipo de dato; no modifica la tabla.
- COALESCE devuelve el primer valor no nulo.
- ROUND, FLOOR y CEILING ayudan con métricas numéricas más presentables.

Ejercicios

1. Convierte amount a decimal con 2 decimales.
2. Convierte release_year a CHAR.
3. Redondea replacement_cost a entero.
4. Usa COALESCE para mostrar return_date y si es NULL usa rental_date.
5. Calcula amount * 1.16 y castea a decimal de 2 posiciones como monto_con_impuesto.
6. Convierte rental_duration a signed integer explícitamente.
7. Usa COALESCE sobre original_language_id para reemplazar NULL por 0.
8. Redondea el promedio de amount por customer_id a 2 decimales.

Solucionario

1.

```
SELECT amount, CAST(amount AS DECIMAL(10,2)) AS amount_decimal FROM payment;
```

2.

```
SELECT title, CAST(release_year AS CHAR) AS anio_texto FROM film;
```

3.

```
SELECT title, replacement_cost, ROUND(replacement_cost, 0) AS replacement_cost_redondeado FROM film;
```

4.

```
SELECT rental_id, COALESCE(return_date, rental_date) AS fecha_referencia FROM rental;
```

5.

```
SELECT payment_id, CAST(amount * 1.16 AS DECIMAL(10,2)) AS monto_con_impuesto FROM payment;
```

6.

```
SELECT title, CAST(rental_duration AS SIGNED) AS rental_duration_int FROM film;
```

7.

```
SELECT film_id, COALESCE(original_language_id, 0) AS original_language_limpio FROM film;
```

8.

```
SELECT customer_id, ROUND(AVG(amount), 2) AS promedio_pago FROM payment GROUP BY customer_id;
```


8. Fechas y tiempos

Fechas son básicas en análisis: cortes mensuales, tendencias, cohortes, horas pico y diferencias entre fechas. Sakila permite practicar con `payment_date`, `rental_date`, `return_date` y `last_update`.

```
SELECT DATE(payment_date) AS fecha,  
YEAR(payment_date) AS anio,  
MONTH(payment_date) AS mes,  
COUNT(*) AS total_pagos  
FROM payment  
GROUP BY DATE(payment_date), YEAR(payment_date), MONTH(payment_date);
```

- DATE extrae solo la fecha; YEAR, MONTH, DAY y HOUR separan componentes.
- DATEDIFF mide días entre fechas.
- DATE_FORMAT sirve para presentar resultados más legibles.

Ejercicios

1. Extrae solo la fecha de `payment_date`.
2. Muestra año, mes y día de `rental_date`.
3. Cuenta pagos por fecha.
4. Suma amount por mes.
5. Calcula los días entre `rental_date` y `return_date`.
6. Muestra la hora de cada pago.
7. Formatea `payment_date` como '%Y-%m'.
8. Cuenta pagos por día de la semana usando DAYOFWEEK.
9. Obtén las rentas devueltas después de 5 días.
10. Muestra pagos ocurridos en la hora 19.

Solucionario

1.

```
SELECT payment_id, DATE(payment_date) AS fecha_pago FROM payment;
```

2.

```
SELECT rental_id, YEAR(rental_date) AS anio, MONTH(rental_date) AS mes, DAY(rental_date) AS dia FROM rental;
```

3.

```
SELECT DATE(payment_date) AS fecha, COUNT(*) AS total_pagos FROM payment GROUP BY DATE(payment_date) ORDER BY fecha;
```

4.

```
SELECT YEAR(payment_date) AS anio, MONTH(payment_date) AS mes, SUM(amount) AS total_pagado FROM payment GROUP BY YEAR(payment_date), MONTH(payment_date) ORDER BY anio, mes;
```

5.

```
SELECT rental_id, DATEDIFF(return_date, rental_date) AS dias_renta FROM rental WHERE return_date IS NOT NULL;
```

6.

```
SELECT payment_id, HOUR(payment_date) AS hora_pago FROM payment;
```

7.

```
SELECT payment_id, DATE_FORMAT(payment_date, '%Y-%m') AS periodo FROM payment;
```

8.

```
SELECT DAYOFWEEK(payment_date) AS dia_semana, COUNT(*) AS total_pagos FROM payment GROUP BY DAYOFWEEK(payment_date) ORDER BY dia_semana;
```

9.

```
SELECT rental_id, rental_date, return_date FROM rental WHERE return_date IS NOT NULL AND  
DATEDIFF(return_date, rental_date) > 5;
```

10.

```
SELECT payment_id, amount, payment_date FROM payment WHERE HOUR(payment_date) = 19;
```

9. Subconsultas y CTEs

Estas técnicas ayudan a dividir problemas complejos. Una subconsulta te deja comparar contra un promedio general o filtrar por resultados intermedios. Una CTE hace la consulta más legible y reusable.

```
WITH pagos_cliente AS (  
  SELECT customer_id, SUM(amount) AS total_pagado  
  FROM payment  
  GROUP BY customer_id  
)  
SELECT *  
FROM pagos_cliente  
WHERE total_pagado > 150;
```

- Usa subconsultas para referencias puntuales.
- Usa CTEs cuando la lógica tenga varias capas.
- En entrevistas, una CTE bien nombrada suele comunicar mejor que una consulta larga y compacta.

Ejercicios

1. Muestra películas con duración mayor al promedio general.
2. Obtén clientes cuyo total pagado sea mayor al promedio del total pagado por cliente.
3. Muestra la categoría con más películas usando subconsulta o CTE.
4. Obtén el cliente con mayor total pagado.
5. Muestra películas cuyo rental_rate sea mayor al promedio de su propio rating.
6. Usa una CTE para contar pagos por día y luego filtra días con más de 150 pagos.
7. Lista actores que participaron en más películas que el promedio por actor.
8. Muestra categorías cuyo ingreso esté por arriba del promedio por categoría.

Solucionario

1.

```
SELECT title, length FROM film WHERE length > (SELECT AVG(length) FROM film);
```

2.

```
WITH pagos_cliente AS (SELECT customer_id, SUM(amount) AS total_pagado FROM payment GROUP BY  
customer_id) SELECT * FROM pagos_cliente WHERE total_pagado > (SELECT AVG(total_pagado) FROM  
pagos_cliente);
```

3.

```
WITH cat_counts AS (SELECT c.name AS categoria, COUNT(*) AS total_peliculas FROM category c JOIN  
film_category fc ON c.category_id = fc.category_id GROUP BY c.name) SELECT * FROM cat_counts ORDER  
BY total_peliculas DESC LIMIT 1;
```

4.

```
WITH pagos_cliente AS (SELECT customer_id, SUM(amount) AS total_pagado FROM payment GROUP BY  
customer_id) SELECT * FROM pagos_cliente ORDER BY total_pagado DESC LIMIT 1;
```

5.

```
SELECT f1.title, f1.rating, f1.rental_rate FROM film f1 WHERE f1.rental_rate > (SELECT  
AVG(f2.rental_rate) FROM film f2 WHERE f2.rating = f1.rating);
```

6.

```
WITH pagos_dia AS (SELECT DATE(payment_date) AS fecha, COUNT(*) AS total_pagos FROM payment GROUP BY  
DATE(payment_date)) SELECT * FROM pagos_dia WHERE total_pagos > 150;
```

7.

```
WITH actor_counts AS (SELECT actor_id, COUNT(*) AS total_peliculas FROM film_actor GROUP BY  
actor_id) SELECT * FROM actor_counts WHERE total_peliculas > (SELECT AVG(total_peliculas) FROM  
actor_counts);
```

8.

```
WITH ingreso_categoria AS (SELECT c.name AS categoria, SUM(p.amount) AS ingreso_total FROM payment p
JOIN rental r ON p.rental_id = r.rental_id JOIN inventory i ON r.inventory_id = i.inventory_id JOIN
film_category fc ON i.film_id = fc.film_id JOIN category c ON fc.category_id = c.category_id GROUP
BY c.name) SELECT * FROM ingreso_categoria WHERE ingreso_total > (SELECT AVG(ingreso_total) FROM
ingreso_categoria) ORDER BY ingreso_total DESC;
```

10. Funciones de ventana

Las ventanas elevan tu nivel. Sirven para rankings, acumulados, promedios móviles y comparaciones dentro de un grupo sin perder el detalle de fila. Son muy útiles para dashboards, segmentaciones y análisis temporal.

```
WITH pagos_cliente AS (  
  SELECT customer_id, SUM(amount) AS total_pagado  
  FROM payment  
  GROUP BY customer_id  
)  
SELECT customer_id,  
total_pagado,  
DENSE_RANK() OVER (ORDER BY total_pagado DESC) AS ranking_pago  
FROM pagos_cliente;
```

- OVER define la ventana.

- PARTITION BY divide por grupo; ORDER BY define el orden dentro del grupo.

- ROW_NUMBER, RANK y DENSE_RANK no se comportan igual con empates.

Ejercicios

1. Haz un ranking de clientes por total pagado.
2. Asigna row_number a películas dentro de cada rating ordenadas por duración descendente.
3. Calcula acumulado diario de amount.
4. Calcula promedio móvil de 3 días sobre total_día.
5. Muestra el top 3 de películas más rentadas por categoría.
6. Compara cada pago contra el pago anterior del mismo cliente usando LAG.
7. Calcula diferencia entre amount y el promedio por cliente usando AVG OVER.
8. Obtén para cada tienda el cliente con mayor total pagado usando ROW_NUMBER.

Solucionario

1.

```
WITH pagos_cliente AS (SELECT customer_id, SUM(amount) AS total_pagado FROM payment GROUP BY  
customer_id) SELECT customer_id, total_pagado, DENSE_RANK() OVER (ORDER BY total_pagado DESC) AS  
ranking_pago FROM pagos_cliente;
```

2.

```
SELECT title, rating, length, ROW_NUMBER() OVER (PARTITION BY rating ORDER BY length DESC) AS rn  
FROM film;
```

3.

```
WITH pagos_dia AS (SELECT DATE(payment_date) AS fecha, SUM(amount) AS total_dia FROM payment GROUP  
BY DATE(payment_date)) SELECT fecha, total_dia, SUM(total_dia) OVER (ORDER BY fecha) AS acumulado  
FROM pagos_dia ORDER BY fecha;
```

4.

```
WITH pagos_dia AS (SELECT DATE(payment_date) AS fecha, SUM(amount) AS total_dia FROM payment GROUP  
BY DATE(payment_date)) SELECT fecha, total_dia, AVG(total_dia) OVER (ORDER BY fecha ROWS BETWEEN 2  
PRECEDING AND CURRENT ROW) AS promedio_movil_3 FROM pagos_dia ORDER BY fecha;
```

5.

```
WITH rentas_categoria AS (SELECT c.name AS categoria, f.title, COUNT(*) AS total_rentas FROM rental  
r JOIN inventory i ON r.inventory_id = i.inventory_id JOIN film f ON i.film_id = f.film_id JOIN  
film_category fc ON f.film_id = fc.film_id JOIN category c ON fc.category_id = c.category_id GROUP  
BY c.name, f.title) SELECT * FROM (SELECT categoria, title, total_rentas, DENSE_RANK() OVER  
(PARTITION BY categoria ORDER BY total_rentas DESC) AS rk FROM rentas_categoria) t WHERE rk <= 3  
ORDER BY categoria, rk, title;
```

6.

```
SELECT customer_id, payment_date, amount, LAG(amount) OVER (PARTITION BY customer_id ORDER BY  
payment_date) AS pago_anterior FROM payment;
```

7.

```
SELECT customer_id, payment_id, amount, AVG(amount) OVER (PARTITION BY customer_id) AS  
promedio_cliente, amount - AVG(amount) OVER (PARTITION BY customer_id) AS diferencia_vs_promedio  
FROM payment;
```

8.

```
WITH pagos_cliente_tienda AS (SELECT c.store_id, p.customer_id, SUM(p.amount) AS total_pagado FROM  
payment p JOIN customer c ON p.customer_id = c.customer_id GROUP BY c.store_id, p.customer_id)  
SELECT * FROM (SELECT store_id, customer_id, total_pagado, ROW_NUMBER() OVER (PARTITION BY store_id  
ORDER BY total_pagado DESC) AS rn FROM pagos_cliente_tienda) t WHERE rn = 1;
```

11. Escenarios reales de negocio

Este bloque transforma Sakila en un laboratorio de análisis. Aunque sea una base de videoclub, las preguntas se parecen a retail, ecommerce o contact center: clientes top, categorías rentables, horas pico, demanda y desempeño por equipo.

```
SELECT DATE_FORMAT(payment_date, '%Y-%m') AS periodo,  
staff_id,  
SUM(amount) AS ventas  
FROM payment  
GROUP BY DATE_FORMAT(payment_date, '%Y-%m'), staff_id  
ORDER BY periodo, ventas DESC;
```

- Piensa en la métrica, la dimensión y el nivel temporal.
- Primero resuelve la lógica, luego embellece alias y formato.
- Haz queries que podrías mandar a un dashboard.

Ejercicios

1. Top 10 clientes por ingreso total.
2. Ventas por staff y por mes.
3. Categorías con mayor ingreso total.
4. Horas con mayor número de pagos.
5. Películas con mayor rotación de renta.
6. Países con más clientes.
7. Duración promedio por categoría.
8. Clientes inactivos que sí han realizado pagos.
9. Promedio de días de renta por categoría.
10. Participación porcentual de ingreso por categoría sobre el total.

Solucionario

1.

```
SELECT customer_id, SUM(amount) AS ingreso_total FROM payment GROUP BY customer_id ORDER BY  
ingreso_total DESC LIMIT 10;
```

2.

```
SELECT DATE_FORMAT(payment_date, '%Y-%m') AS periodo, staff_id, SUM(amount) AS ventas FROM payment  
GROUP BY DATE_FORMAT(payment_date, '%Y-%m'), staff_id ORDER BY periodo, ventas DESC;
```

3.

```
SELECT c.name AS categoria, SUM(p.amount) AS ingreso_total FROM payment p JOIN rental r ON  
p.rental_id = r.rental_id JOIN inventory i ON r.inventory_id = i.inventory_id JOIN film_category fc  
ON i.film_id = fc.film_id JOIN category c ON fc.category_id = c.category_id GROUP BY c.name ORDER BY  
ingreso_total DESC;
```

4.

```
SELECT HOUR(payment_date) AS hora, COUNT(*) AS total_pagos FROM payment GROUP BY HOUR(payment_date)  
ORDER BY total_pagos DESC;
```

5.

```
SELECT f.title, COUNT(*) AS total_rentas FROM rental r JOIN inventory i ON r.inventory_id =  
i.inventory_id JOIN film f ON i.film_id = f.film_id GROUP BY f.title ORDER BY total_rentas DESC  
LIMIT 20;
```

6.

```
SELECT co.country, COUNT(*) AS total_clientes FROM customer cu JOIN address a ON cu.address_id =  
a.address_id JOIN city ci ON a.city_id = ci.city_id JOIN country co ON ci.country_id = co.country_id  
GROUP BY co.country ORDER BY total_clientes DESC;
```

7.

```
SELECT c.name AS categoria, ROUND(AVG(f.length), 2) AS duracion_promedio FROM film f JOIN
film_category fc ON f.film_id = fc.film_id JOIN category c ON fc.category_id = c.category_id GROUP
BY c.name ORDER BY duracion_promedio DESC;
```

8.

```
SELECT cu.customer_id, cu.first_name, cu.last_name, SUM(p.amount) AS total_pagado FROM customer cu
JOIN payment p ON cu.customer_id = p.customer_id WHERE cu.active = 0 GROUP BY cu.customer_id,
cu.first_name, cu.last_name ORDER BY total_pagado DESC;
```

9.

```
SELECT c.name AS categoria, ROUND(AVG(DATEDIFF(r.return_date, r.rental_date)), 2) AS
promedio_dias_renta FROM rental r JOIN inventory i ON r.inventory_id = i.inventory_id JOIN
film_category fc ON i.film_id = fc.film_id JOIN category c ON fc.category_id = c.category_id WHERE
r.return_date IS NOT NULL GROUP BY c.name ORDER BY promedio_dias_renta DESC;
```

10.

```
WITH ingreso_categoria AS (SELECT c.name AS categoria, SUM(p.amount) AS ingreso_total FROM payment p
JOIN rental r ON p.rental_id = r.rental_id JOIN inventory i ON r.inventory_id = i.inventory_id JOIN
film_category fc ON i.film_id = fc.film_id JOIN category c ON fc.category_id = c.category_id GROUP
BY c.name), total AS (SELECT SUM(ingreso_total) AS total_general FROM ingreso_categoria) SELECT
ic.categoria, ic.ingreso_total, ROUND(ic.ingreso_total / t.total_general * 100, 2) AS pct_total FROM
ingreso_categoria ic CROSS JOIN total t ORDER BY ic.ingreso_total DESC;
```


12. Buenas prácticas para sonar pro

Ser pro no es solo conocer sintaxis. Es escribir SQL mantenible, entendible y verificable. Un buen analista explica de dónde sale una métrica, nombra bien sus CTEs y evita consultas innecesariamente crípticas.

```
WITH pagos_cliente AS (  
  SELECT customer_id,  
    SUM(amount) AS total_pagado,  
    COUNT(*) AS total_pagos  
  FROM payment  
  GROUP BY customer_id  
)  
SELECT customer_id,  
  total_pagado,  
  total_pagos,  
  ROUND(total_pagado / total_pagos, 2) AS ticket_promedio  
FROM pagos_cliente  
ORDER BY total_pagado DESC;
```

- Una CTE con nombre claro comunica más que una subconsulta anidada sin alias.
- No abusos de SELECT * en producción.
- Valida nulos, duplicados y granularidad antes de presentar resultados.

Ejercicios

1. Reescribe una consulta agregada usando CTE y alias claros.
2. Crea una consulta con métricas de cliente: total pagado, número de pagos y ticket promedio.
3. Haz una versión legible del top de películas rentadas por categoría usando CTEs.
4. Agrega comentarios de línea a una consulta compleja en tu notebook para explicar cada bloque.
5. Crea una consulta que use COALESCE, CASE y ROUND en una sola salida.
6. Escribe una consulta con filtros por fecha y agrega un alias de periodo claro.
7. Haz una validación simple de duplicados por payment_id.
8. Crea tu propia consulta de dashboard con al menos una métrica, una dimensión y una fecha.

Solucionario

1.

```
WITH pagos_cliente AS (SELECT customer_id, SUM(amount) AS total_pagado FROM payment GROUP BY  
customer_id) SELECT customer_id, total_pagado FROM pagos_cliente ORDER BY total_pagado DESC;
```

2.

```
WITH pagos_cliente AS (SELECT customer_id, SUM(amount) AS total_pagado, COUNT(*) AS total_pagos FROM  
payment GROUP BY customer_id) SELECT customer_id, total_pagado, total_pagos, ROUND(total_pagado /  
total_pagos, 2) AS ticket_promedio FROM pagos_cliente;
```

3.

```
WITH rentas_pelicula_categoria AS (SELECT c.name AS categoria, f.title, COUNT(*) AS total_rentas  
FROM rental r JOIN inventory i ON r.inventory_id = i.inventory_id JOIN film f ON i.film_id =  
f.film_id JOIN film_category fc ON f.film_id = fc.film_id JOIN category c ON fc.category_id =  
c.category_id GROUP BY c.name, f.title) SELECT categoria, title, total_rentas FROM  
rentas_pelicula_categoria ORDER BY categoria, total_rentas DESC;
```

4.

```
-- Ejercicio libre: documenta tus CTEs, joins y filtros en una consulta existente.
```

5.

```
SELECT payment_id, ROUND(amount, 2) AS monto, CASE WHEN amount >= 5 THEN 'Alto' ELSE 'Bajo' END AS  
rango, COALESCE(CAST(staff_id AS CHAR), 'NA') AS staff_txt FROM payment;
```

6.

```
SELECT DATE_FORMAT(payment_date, '%Y-%m') AS periodo, SUM(amount) AS ventas FROM payment WHERE  
payment_date >= '2005-05-01' GROUP BY DATE_FORMAT(payment_date, '%Y-%m');
```

7.

```
SELECT payment_id, COUNT(*) AS repeticiones FROM payment GROUP BY payment_id HAVING COUNT(*) > 1;
```

8.

-- Ejercicio abierto: por ejemplo ventas por staff y hora, clientes por país, rentas por categoría y mes.

Plan sugerido de estudio

Semana 1: módulos 1 a 3. Enfócate en sintaxis, filtros, patrones y limpieza de salida.

Semana 2: módulos 4 a 6. Prioriza agregaciones, HAVING y joins.

Semana 3: módulos 7 a 9. Domina casting, fechas, subconsultas y CTEs.

Semana 4: módulos 10 a 12. Practica ventanas, escenarios reales y consultas limpias.

Reto final

Construye un mini dashboard en SQL con cinco preguntas:

- Ingresos por mes y por staff.
- Top 10 clientes por total pagado y ticket promedio.
- Categorías con mayor ingreso y su participación porcentual.
- Películas con mayor rotación de renta por categoría.
- Horas y días de la semana con mayor actividad de pago.

Después, intenta explicar cada consulta como si fueras a presentarla en una entrevista técnica o a tu jefe.

Fin de la guía.